JWT 게시판 구현 보고서

01 서비스 소개와 구현 범위

학번 202219377 이름: 김지후 제출일: 2026/10/28

서비스 이름 음악 감상 게시판

기술 조합과 버전 React 19.3.0 / Spring Boot 3.5.16 / OpenJDK 21.0.12.1, Node.js 24.21.0

GitHub 저장소 URL **https://github.com/jikim08/wsd-jwt-board**

소스 코드 기준 브랜치: main / SHA: b1ef32c86b164304c2b8f038241ed7c9df3ba9e1

음악 감상 게시판은 음악을 듣고 느낀 감상을 기록하고 공유하고 싶은 사용자를 위한 서비스이다. 게시글 제목에는 곡명과 가수명을 적을 수 있으며, 본문에는 음악에 대한 개인적인 감상을 작성할 수 있다.
로그인한 사용자는 글을 작성할 수 있으며, 자신의 감상평에 대해서만 수정, 삭제할 수 있다.

실무에서 널리 활용되는 Spring Boot 와 React 를 선택하여 미리 현업에서 더 많이 쓰게 될 프레임워크를 접해보고 싶었다. 제공 템플릿의 게시판 CRUD 와 SQLite 저장구조를 활용하면서 JWT 발급, 검증, 작성자 권한 검사, 프론트 로그인 연결 및 오류 처리를 추가하였다.

기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	완료	BoardController.java, PostStore.java, api.js, main.jsx	비로그인 상태에서도 조회 가능
로그인과 JWT 발급	완료	AuthService.java, BoardController.java, api.js	제공 계정 검증 후 HS256 JWT 발급
인증과 내 정보 조회	완료	AuthService.java, BoardController.java	서버 권한 검사 및 작성자별 버튼 표시
본인 글 작성·수정·삭제	완료	AuthService.java, BoardController.java, api.js, main.jsx	서버 권한 검사 및 작성자별 버튼 표시
만료·로그아웃·오류 표시	완료	api.js, main.jsx	401:로그인 해제, 403: 로그인 유지, 404: 목록 갱신
DB 데이터 유지	완료	PostStore.java, DatabaseConfig.java	SQLite 파일 저장 및 서버 재시작 후 유지 확인

웹서비스설계 02

구조와 변경 내역

React 가 화면의 입력 상태를 관리하고, api.js 가 서버와의 통신을 담당한다. 브라우저의 /api 요청은 Vite 프록시를 통해 Spring Boot 서버로 전달된다. 서버에서는 BoardController 가 요청을 받아 AuthService 로 인증/권한을 확인하고, PostStore 를 통해 SQLite 의 게시글을 조회하거나 변경한다. 테스트 계정은 Users 가 메모리에 준비하며, 게시글을 SQLite 파일에 저장한다.

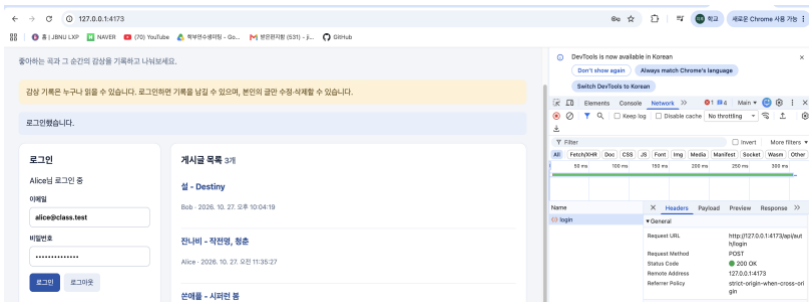
로그인 요청: 사용자가 이메일과 비밀번호를 입력하면 login()이 POST /api/auth/login 을 호출한다. 서버는 입력값과 계정을 확인하고 JWT 와 사용자 정보를 반환한다. 프론트는 이를 api.js 의 session 변수에 저장하고 로그인 사용자 표시를 갱신한다.

게시글 수정 요청: api.update()가 수정내용을 전달하면 request()가 Authorization: Bearer [JWT] 헤더를 붙인다. 서버는 토큰을 검증하고, 게시글의 존재를 확인하고, 작성자 권한을 검사하고, 입력값을 검증하는 순서를 거쳐 DB 를 수정한다. 성공하면 프론트가 상세 화면과 목록을 갱신한다.

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
AuthService.java 파일 생성자, issueToken()	설정 주입 및 미완성 토큰 발급	키 형식, TTL 검사, 필수 클레임 설정, HS256 서명 (AI 초안 후 수정)	유효한 설정으로 JWT 발급
AuthService.java currentUserId(), 보조 함수	데모 사용자 ID 1 반환	Bearer 추출, 서명,알고리즘,클레임,실제 계정 검증(AI 초안 후 수정)	검증된 사용자만 요청 처리에 사용
AuthService.java 파일 assertOwner()	권한 검사 없음	인증된 사용자 ID 와 DB 의 작성자 ID 비교 (본인 구현)	타인의 수정 삭제를 403 으로 차단
frontend-react/src/api.js	로그인 토큰 전송 미완성	메모리 세션 저장, 인증 헤더 첨부, 401 세션 해제, canEdit() 구현 (AI 작성, 본인 작성 혼합)	서버 인증과 화면 상태 연결
frontend-react/src/main.jsx	재공 화면과 공통 작업 처리	세션과 사용자 표시 동기화, 로그아웃 버튼 상태, 404 시 상태 초기화 및 목록 갱신 (AI 초안 후 수정)	요청 결과에 맞게 화면 상태 유지

로컬 실행 기록

개발 환경은 macOS 26.4.1, OpneJDK 21.0.12.1, Node.js 24.21.0, npm 11.19.0 이다. 저장소를 별도 폴더에 클론한 검증 환경에서 npm ci --ignore-scripts 로 프론트 의존성을 설치하고 ./mvnw -q compile 로 백엔드 컴파일 성공을 확인했다.



벡엔드는 JWT_SECRET 에 64 자리 16 진수 키를 환경변수로 설정했고, 발급자는 webservice-class, 사용대상은 board-api, 기본 토큰 유효시간은 90 초로 설정했다. 이후 ./mvnw spring-boot:run 을 실행해 http://127.0.0.1:8000/api/health 에서 정상 응답을 확인했다. 게시글은 backend-spring/data/board.splite 에 저장되고, 서버를 종료하고 다시 실행해도 기존 게시글이 유지됨을 확인했다.

프론트는 frontend-react 에서 npm ci 로 의존성을 설치하고 npm run dev 로 개발 중 동작을 확인했다. 최종 실행 확인에서는 개발 서버를 종료한 뒤 npm run build 로 빌드하여 dist 폴더를 생성하고, npm run preview 를 실행했다. http://127.0.0.1:4173 에서 게시글 목록 조회와 로그인이 정상 동작하여 빌드된 프론트와 8000 번 백엔드의 API 연결을 확인했다.

웹서비스설계 03

핵심 코드 설명

3 1 로그인과 JWT 발급

파일 / 함수:

AuthService.java / issueToken()

코드 발췌

```
Instant now = Instant.now(); // 발급 시점의 현재 시각을 구함


  return Jwts.builder()
    .subject(Long.toString(user.id()))        //사용자 id 설정
    .issuer(issuer)                           //발급자 설정
    .audience().add(audience).and()           //사용 대상 지정

    .expiration(Date.from(now.plusSeconds(ttl)))  //현재 시각에 유효시간 ttl 을 더해 만료 시각을 설정

    .signWith(key, Jwts.SIG.HS256)            //비밀키로 서명, 알고리즘 지정
    .compact();                               //header.payload.signature 형태의 JWT 문자열 반환
```

동작과 응답 코드 설명

제공된 BoardController.login()은 이메일과 비밀번호를 받아 입력 필드와 문자열 형식, 길이를 검사하고, Users.verify()로 계정을 확인한다. 입력이 잘못되면 HTTP 400 과 JSON code BAD_INPUT, 계정 정보가 일치하지 않으면 HTTP 401 과 INVALID_CREDENTIALS 를 반환한다.

직접 구현한 AuthService.issueToken()은 계정 확인을 통과한 사용자 정보를 받아 JWT 를 발급한다. sub 에는 사용자 ID 문자열, iss 에는 발급자, aud 에는 사용 대상을 저장한다. exp 는 현재 시각에 설정된 유효 시간(ttl 90 초)을 더해 지정한다. 서버의 비밀 키로 HS256 서명을 생성하고 compact()로 JWT 문자열을 반환한다.

로그인 성공시 HTTP 200 으로 accessToken, expiresIn, user 를 반환한다.

3 2 브라우저 토큰 저장과 전송

파일 / 함수: api.js / login(), authorizationHeaders()

코드 발췌

```
let session = { accessToken: "", user: null };
```

```
export async function login(email, password) {
// request()를 이용할 수 있습니다. 성공하면 user 를 반환하세요.
  const data = await request("/auth/login", {
    method: "POST",
    body: JSON.stringify({ email, password }),
  });

  session = {
    accessToken: data.accessToken,
    user: data.user,
  }

  return session.user;
}
function authorizationHeaders() {
  if(!session.accessToken) {
    return {}
  }

  return {
    Authorization : `Bearer ${session.accessToken}`
  };
}
```

동작과 응답 코드 설명

login()은 이메일과 비밀번호로 로그인 요청을 보내고, HTTP 200 으로 받은 토큰과 사용자 정보를 session 변수에 저장한다. 토큰은 메모리에만 보관하며 localStorage 와 sessionStorage 에는 저장하지 않는다.

이후 공통 요청 함수인 request()가 authorizationHeaders()를 호출하여 토큰을 Authorization: Bearer [JWT] 형식으로 전송한다. 토큰이 없으면 인증 헤더를 붙이지 않는다.

로그아웃 시 clearSession()이 토큰과 사용자 정보를 비운다. API 응답이 HTTP 401 인 경우에도 세션을 비우고 오류를 표시하며, HTTP 403 인 경우에는 로그인 상태를 유지한다. 페이지를 새로고침하면 메모리 변수가 초기화되어 로그인 상태가 해제된다.

웹서비스설계 03 계속

서버 인증과 작성자 권한

3 3 서버 JWT 검증

파일 / 함수: AuthService.java / extractToken(), verifyToken(), currentUserId()

코드 발췌

```
try{
    Claims claims = Jwts.parser()
      .verifyWith(key)                    // 발급할때 사용한 비밀키로 서명 검증
      .sig().clear().add(Jwts.SIG.HS256).and()  //허용 알고리즘 HS256 으로 제한

      .requireIssuer(issuer)              // 발급자와 사용 대상이 서버 설정과 일치해야함
      .requireAudience(audience)

      .build()                            //JwtParser 빌드
      .parseSignedClaims(token)           //검증
      .getPayload();                      //검증 통과한 payload 를 꺼냄
    if (claims.getExpiration() == null){
      throw new ApiError(401, "INVALID_TOKEN", "만료 시각이 없는 토큰입니다.");
    }
      return claims;
  } catch (ExpiredJwtException e) {
    throw new ApiError(401, "TOKEN_EXPIRED", "로그인이 만료되었습니다.");
  } catch (JwtException | IllegalArgumentException e){
    throw new ApiError(401, "INVALID_TOKEN", "유효하지 않은 토큰입니다.");
  }
```

동작과 응답 코드 설명

extractToken()은 요청의 Authorization 헤더에서 Bearer 토큰을 추출한다. 헤더가 없거나 형식이 잘못되면 HTTP 401 과 JSON code TOKEN_REQUIRED 를 반환한다.

verifyToken()은 HS256 만 허용하고, 서명·발급자(iss)·사용 대상(aud)·만료 시각(exp)을 검증한다. 만료 시각의 존재 여부도 별도로 확인한다. 만료된 토큰은 HTTP 401 TOKEN_EXPIRED, 서명이나 클레임이 잘못된 토큰은 HTTP 401 INVALID_TOKEN 으로 응답한다.

currentUserId()는 검증된 클레임의 sub 가 양의 정수 문자열이고 long 범위에 들어가는지 검사한다. 이후 실제 계정이 존재하는지 확인하여 사용자 ID 를 반환한다. 이 검사에 실패하면 HTTP 401 INVALID_TOKEN 을 반환한다. 검증에 성공한 ID 는 내 정보 조회와 게시글 작성·수정·삭제 처리에 사용한다.

3 4 작성자 권한 검사

파일 / 함수: AuthService.java / assertOwner()

코드 발췌

```
public void assertOwner(long userId, long authorId) {
  // TODO BE-03: 다른 작성자이면 403 FORBIDDEN. 현재는 검사하지 않습니다.
  if (userId != authorId){
    throw new ApiError(403, "FORBIDDEN", "본인이 작성한 글만 변경할 수 있습니다.");
  }
 }
```

동작과 응답 코드 설명

assertOwner()는 JWT 검증으로 얻은 사용자 ID 와 DB 에 저장된 게시글의 작성자 ID 를 비교한다. 제공된 수정·삭제 라우트에서 데이터 변경 전에 호출하며, 두 ID 가 다르면 HTTP 403 과 JSON code FORBIDDEN 을 반환한다. 작성자가 일치하면 다음 처리를 진행하고, 수정 성공은 HTTP 200, 삭제 성공은 본문 없는 HTTP 204 로 응답한다.

HTTP 401 은 토큰이 없거나 유효하지 않아 인증에 실패한 경우이고, HTTP 403 은 인증에는 성공했지만 해당 글을 변경할 권한이 없는 경우이다.

작성자는 요청 본문이 아닌 검증된 사용자 ID 로 결정한다. 본문에 authorId 나 userId 를 추가하면 제공된 입력 검증에서 HTTP 400 BAD_INPUT 으로 거부한다. 프론트의 수정·삭제 버튼을 숨기는 것과 별개로 서버에서도 권한을 검사하여 직접 API 를 호출하는 경우를 막는다.

웹서비스설계 04

테스트 결과

실행 환경: macOS 26.4.1, OpneJDK 21.0.12.1, Node.js 24.21.0, npm 11.19.0, Chrome 브라우저

로컬 배포 확인: 3 페이지 로컬 실행 기록 참고

테스트 일시 및 코드 버전: 10 월 28 일 b1ef32c86b164304c2b8f038241ed7c9df3ba9e1

토큰 TTL: 일반 실행 90 초 / 만료 검증 10 초

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T01	무토큰으로 목록과 기존 글 상세조회	각각 200	목록 200 /해당 없음 상세 200 / 해당 없음	Pass	screenshot/T01_*
T02	Alice 로그인	200, JWT, 공개 사용자 정보와 로그인 표시	200 / 해당없음	Pass	screenshot/T02_*
T03	잘못된 비밀번호와 없는 이메일로 각각 로그인	401 INVALID_CREDENTIALS	두 경우 모두 401/INVALID_CREDENTIALS	Pass	screenshot/T03_*
T04	무토큰 /me 조회와 글 등록	401 TOKEN_REQUIRED	두 경우 모두 401/TOKEN_REQUIRED	Pass	screenshot/T04_*
T05	Alice 토큰으로 글 작성 후 /me 조회	201, authorId 1 / 200, Alice 정보	글 등록: 201/해당없음 /me: 200/해당없음	Pass	screenshot/T05_*
T06	Alice 본인 글 수정, 삭제	200/204	수정 200 / 해당없음 삭제 204 / 해당없음	Pass	screenshot/T06_*
T07	Bob 토큰으로 Alice 글 수정	403 FORBIDDEN	403 / FORBIDDEN	Pass	screenshot/T07_*
T08	Bob 토큰으로 Alice 글 삭제	403 FORBIDDEN	403 / FORBIDDEN	Pass	screenshot/T08_*

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T09	변조 토큰으로 /me, 작성, 수정, 삭제	401 INVALID_TOKEN	401 / INVALID_TOKEN	Pass	screenshot/T09_*
T10	실제 만료 후 /me 와 게시글 변경	401 TOKEN_EXPIRED. 로그인 해제	모두 401/TOKEN_EXPIRED 로그인 해제	Pass	screenshot/T10_*
T11	로그아웃 후 /me 조회	401 TOKEN_REQUIRED	401/TOKEN_REQUIRED	Pass	screenshot/T11_*
T12	Bob 등록 요청에 authorId: 1 과 userId: 1 을 각각 추가	400 BAD_INPUT	모두 400/BAD_INPUT	Pass	screenshot/T12_*
T13	비거나, 공백이거나, 길이 초과한 제목/본문	400 BAD_INPUT	모두 400/BAD_INPUT	Pass	screenshot/T13_*
T14	없는 글 조회, 수정, 삭제, 잘못된 ID 조회	없는 글 404 NOT_FOUND 잘못된 ID 400 BAD_INPUT	없는 글 404/NOT_FOUND 잘못된 ID 400/BAD_INPUT	Pass	screenshot/T014_*
T15	글 작성 후 실제 서버 종료, 재실행	기존 글과 작성자,내용 유지	기존과 동일한 글 ID, 제목, 본문, 작성자가 유지됨	Pass	screenshot/T15_*
T16	로그인 후 페이지 새로고침	공개 글 유지, 로그인 해제	공개 글과 목록은 유지되고, 로그인 해제	Pass	screenshot/T016_*
T17	무토큰 수정, 삭제와 Bob 본인 작성, 수정, 삭제	무토큰 401, 본인 작업 201 200 204	무토큰 401/TOKEN_REQUIRED bob 본인 글 201, 200, 204	Pass	screenshot/T17_*

웹서비스설계 04 계속

HTTP 응답 확인 기록

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
정상 조회·로그인·수정	**200**	200/해당없음	T01 T02 T06 T07
새 글 등록	**201 및 Location**	201/해당없음	T05
본인 글 삭제	**204 및 빈 본문**	204/해당없음	T06 T17

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
입력 오류·추가 작성자 필드	**400 BAD_INPUT**	400/BAD_INPUT	T12 T13
잘못된 계정	**401 INVALID_CREDENTIALS**	401/INVALID_CREDENTIALS	T03
토큰 없음·헤더 형식 오류	**401 TOKEN_REQUIRED**	401/TOKEN_REQUIRED	T04 T11 T17
변조·클레임 오류	**401 INVALID_TOKEN**	401/INVALID_TOKEN	T09
정상 서명의 만료 토큰	**401 TOKEN_EXPIRED**	401/TOKEN_EXPIRED	T10
다른 작성자의 변경	**403 FORBIDDEN**	403/FORBIDDEN	T07 T08
없는 글	**404 NOT_FOUND**	404/NOT_FOUND	T14

401 응답 헤더: 보호 API 의 응답에서 WWW-Authenticate: Bearer 를 확인했다. 관련 증거는 T04, T09, T10 응답 헤더 캡쳐에 기록되어 있다.

401 과 403 의 화면 처리: 401 응답에서는 로그인 상태를 해제하고 로그인 필요 또는 만료 안내를 표시했다. 403 응답에서는 로그인 상태를 유지하며 권한 부족을 안내했다.

변경 실패 후 데이터 유지: T07·T08 에서는 수정·삭제 거부 후 공개 상세 조회로 원문이 유지됨을 확인했다. T09 에서는 변조 토큰 요청 전후의 게시글 목록을 비교하여 데이터 유지 결과가 true 임을 확인했다.

없는 글에 대한 오류 우선순위: 정상 토큰으로 없는 글을 수정한 요청은 T14 에서 404 / NOT_FOUND 를 확인했다. 토큰 없이 같은 없는 글을 수정하는 조합은 별도 확인 후 실제 응답을 기록한다.

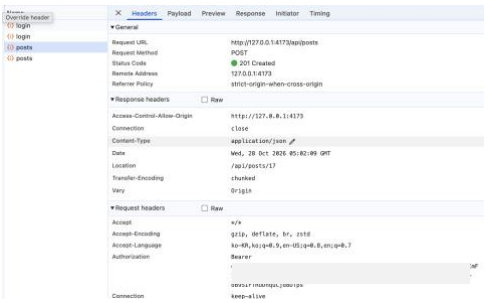
웹서비스설계 04 계속

테스트 증거

로그인과 토큰 전송

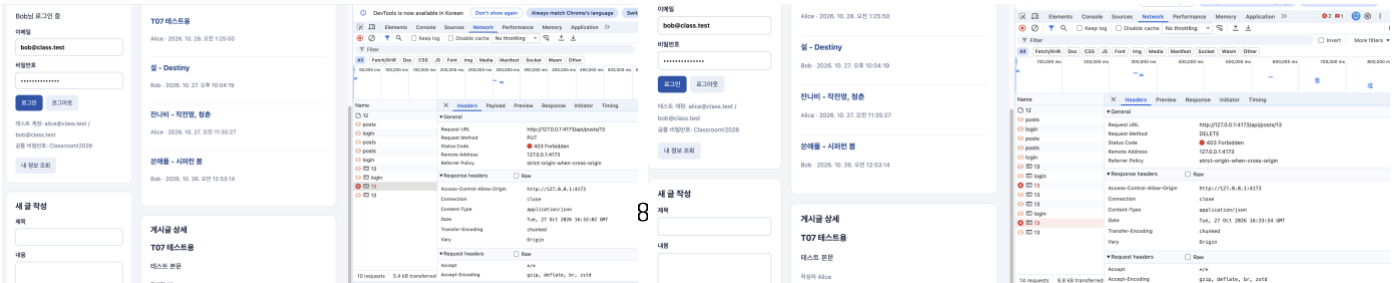
T02 로그인 성공 후 Response 화면과 T05 게시글 등록 요청에 토큰이 첨부됨





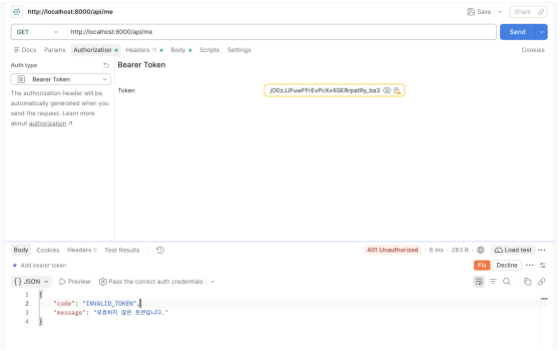
다른 작성자의 수정과 삭제

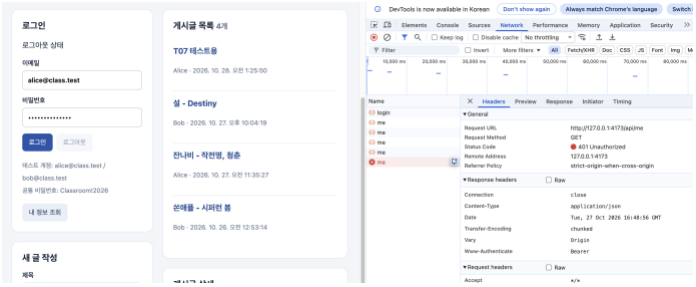
T07 수정 거부 및 원문이 유지됨, T08 삭제 거부 및 원문이 유지됨



토큰 변조와 만료

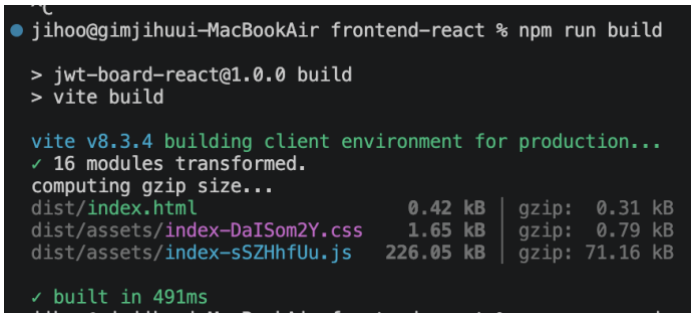
T09 JWT 서명을 변조한 요청은 HTTP 401 과 IINVALID_TOKEN 으로 거부, T10 만료된 토큰은 401 TOKEN_EXPIRED 반환

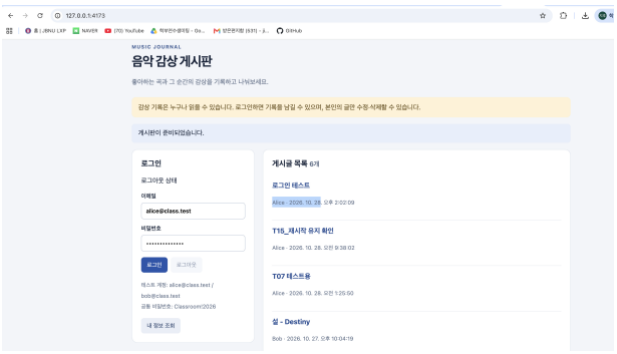




자동 검사와 로컬 배포 증거

프론트 빌드 성공 후 4173 번 화면에서 게시글 목록 조회와 로그인이 정상 동작했다. 백엔드를 실제로 종료하고 다시 실행한 뒤에도 동일한 글 ID·제목·본문·작성자가 유지되었다





추가적인 캡쳐 증거들은 sceenshot 폴더 안에 T01~T17 번까지 구분하여 담아두었다.

웹서비스설계 05

AI 활용과 참고 자료

AI 사용 여부: 사용

사용하지 않았다면 미사용이라고 명시하고 참고한 수업 자료·공식 문서의 출처를 작성하세요. 사용했다면 전체 사용 범위를 요약하고 대표 사례 2~3 건을 작성하세요. 사용이 1 건이면 그 1 건만 적습니다. 대화 전체 제출은 필요하지 않습니다.

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식
GPT 6.1 Sol	학습/생성	AuthService.java api.js	설명을 먼저 확인하고 제안 코드를 검토하여 과제 요구사항과 대조하며 적용.
GPT 6.1 Sol	문장 정리	보고서 전반	초안을 먼저 작성하고 이를 AI 에게 넘겨 빠진 부분이나 적절하지 않은 표현, 다듬은 문장을 추천받아 이를 검토하여 선별적으로 적용

대표 사례

도구 / 날짜: ChatGPT / 2026 년 10 월 25 일

요청한 내용: 과제의 API 명세와 구현 체크리스트에 맞춰 인증 코드를 단계별로 제안하도록 요구했다. 코드만 제시하지 않고 작업 목적, 사용하는 개념, 각 함수의 역할을 설명하며 주석을 포함하도록 요구했다.

AI 가 제안한 내용: JWT 를 이용한 HS256 토큰 발급 코드와 Bearer 토큰 추출, 서명과 클레임 검증, 실제 사용자 확인 코드를 제안했다.

실제 반영 위치: AuthService.java 의 생성자, issueToken(), extractToken(), verifyToken(), currentUserId()

내가 확인하고 수정한 내용: builder(), parser(), compact(), build()의 역할과 클레임 설정 방법을 질문하여 확인했다. Spring 의 설정값 주입 방식과 사용자 객체가 전달되는 흐름도 확인했다. 제안된 코드를 단계별로 반영하고 실행 환경에 맞게 JWT 키와 설정값을 지정했다. 제공된 컨트롤러와 저장소 코드는 유지했다.

검증 방법과 결과: 정상 로그인과 내 정보 조회를 확인했다. 토큰이 없는 요청, 서명을 변조한 요청, 만료된 토큰 요청이 각각 지정된 오류로 거부되는지 확인했다. Bob 이 Alice 의 글을 수정하거나 삭제하는 요청도 403 으로 거부되고 기존 데이터가 유지되는지 확인했다. 결과는 T02, T04, T05, T07 부터 T10 에 기록했다.

대표 사례 2

도구 / 날짜: ChatGPT / 2026 년 10 월 26 일

요청한 내용: 구현 체크리스트를 기준으로 현재 단계에서 필요한 로그인 상태 처리를 제안하도록 요구했다. 토큰과 사용자 정보를 어디에 저장하고, 요청에 어떻게 전달하며, 오류가 발생했을 때 화면을 어떻게 갱신하는지 설명하도록 요구했다.

AI 가 제안한 내용: 토큰을 모듈의 메모리에 저장하고 공통 요청 함수에서 Authorization 헤더에 첨부하는 코드를 제안했다. 401 응답에서는 세션을 지우고, 403 응답에서는 로그인 상태를 유지하도록 처리했다. API 모듈의 세션을 React 의 사용자 표시와 동기화하는 코드도 제공했다.

실제 반영 위치: api.js 의 login(), authorizationHeaders(), request(), clearSession(), canEdit()와 main.jsx 의 run(), 로그아웃 처리.

내가 확인하고 수정한 내용: clearSession()이 호출되는 위치와 오류 메시지가 화면에 전달되는 과정을 확인했다. 공통 요청 함수의 헤더 구성과 비동기 처리 흐름도 질문하며 코드를 읽었다. 제안된 코드를 반영한 뒤 localStorage 와 sessionStorage 에 토큰이 저장되지 않는지 확인했다.

검증 방법과 결과: 로그인 성공과 실패 안내, 만료 시 로그인 해제, 권한 부족 시 로그인 유지, 로그아웃과 새로고침 동작을 확인했다. Alice 와 Bob 의 글에서 수정과 삭제 버튼의 표시 상태도 확인했다. 관련 결과는 T02, T03, T07, T10, T11, T16 에 기록했다.

참고 자료

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
docs/02_API 명세.md	인증, 권한 검사, 입력 검증	JWT 필수 클레임, HTTP 상태, JSON code, 응답 헤더 기준
docs/03_구현체크리스트.md	구현 진행 관리	백엔드와 프론트의 구현 순서와 확인 항목
docs/05_테스트시나리오.md	보고서 4 장	T01~T017 까지의 테스트 절차와 기대 결과
docs/07_로컬실행안내.md	설치와 로컬 실행	환경변수 설정, 개발 서버 실행, 빌드 결과 확인 절차
5 주차, 6 주차 강의 자료	JWT 인증과 작성자 권한 검사 구현	JWT 의 구조와 발급 및 검증 과정, 인증과 권한 검사의 개념

웹서비스설계 06

이해도 확인과 회고

1. 요청 본문의 authorId 를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

요청 본문의 authorId 를 그대로 사용하면 Bob 이 Alice 의 ID 를 넣어 다른 사람 명의로 글을 작성할 수 있다. 내 코드에서는 currentUserId()가 JWT 를 검증하여 얻은 사용자 ID 를 작성자로 사용한다. 제공된 BoardController 의 입력 검증은 title 과 content 만 허용하므로 authorId 나 userId 를 추가한 요청은 400 BAD_INPUT 으로 거부한다. T12 에서 두 필드를 각각 추가하여 요청했을 때 글이 생성되지 않는 것을 확인했다.

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

버튼이 보이지 않아도 Console 이나 API 클라이언트에서 수정과 삭제 요청을 직접 보낼 수 있다. 따라서 프론트의 canEdit()과 별도로 서버에서 권한을 검사해야 한다.

서버의 assertOwner()는 JWT 검증으로 얻은 사용자 ID 와 DB 에 저장된 게시글 작성자 ID 를 비교한다. 두 값이 다르면 데이터를 변경하기 전에 403 FORBIDDEN 을 반환한다. T07 과 T08 에서 Bob 의 토큰으로 Alice 의 글에 직접 요청을 보내 수정과 삭제가 거부되고 원문이 유지되는 것을 확인했다.

3. JWT decode 와 verify 의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

decode 는 토큰에 담긴 내용을 읽는 것이고, verify 는 서명을 검증하여 토큰이 유효한지 확인하는 것이다. 내용을 읽을 수 있다는 이유만으로 해당 토큰을 신뢰할 수는 없다.

verifyToken()에서는 허용 알고리즘을 HS256 으로 제한하고 서버 키로 서명을 검증한다. 발급자와 사용 대상이 설정값과 일치하는지, 만료 시각이 존재하고 아직 지나지 않았는지도 확인한다. currentUserId()에서는 sub 가 양의 정수 문자열인지, long 범위에 들어가는지, 실제 계정이 존재하는지 추가로 검사한다. 검증을 통과한 사용자 ID 만 요청 처리에 사용한다.

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

로그아웃은 현재 탭의 메모리에 저장된 토큰과 사용자 정보를 지우고 화면을 로그아웃 상태로 바꾼다.

토큰 만료는 exp 에 설정된 시각이 지난 상태이다. 만료된 토큰으로 보호 API 에 요청하면 서버가 401 TOKEN_EXPIRED 를 반환하고, 프론트가 로그인 상태를 해제하며 재로그인을 안내한다.

새로고침은 프론트의 메모리 변수를 초기화하므로 로그인 상태가 해제된다. 게시글은 SQLite 에 저장되어 있어 그대로 유지된다.

현재 구현에는 토큰 폐기 목록이 없으므로 로그아웃해도 이미 복사된 토큰 자체가 무효화되지는 않는다. 복사된 토큰은 유효 시간이 남아 있으면 사용할 수 있고, 만료되면 서버에서 거부된다.

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

서버에서 게시글 없음 오류를 반환했을 때 화면에 남은 상세 정보와 수정 상태를 정리하는 부분이 어려웠다. 처음에는 API 오류를 메시지로 표시했지만, 선택한 글과 수정 중인 입력값을 초기화하거나 목록을 다시 불러오는 처리가 없었다.

원인을 확인하기 위해 api.js 의 request()가 오류를 전달하는 과정과 main.jsx 의 run()이 이를 처리하는 과정을 살펴보았다. 이후 run()에서 HTTP 상태가 404 이고 code 가 NOT_FOUND 인 경우를 구분하여 선택한 글을 비우고, reset()으로 수정 상태와 입력값을 초기화한 뒤 refresh()로 목록을 갱신하도록 수정했다.

같은 계정으로 두 탭을 열어 한 탭에서 글을 삭제한 뒤 다른 탭에서 수정을 시도하는 방식으로 재검증했다. 오류 안내가 표시되고 상세 화면과 수정 상태가 초기화되며 목록이 갱신되는 것을 확인했다.

현재 남은 문제와 다음에 개선할 점:

필수 기능에 대한 수동 테스트를 완료했다. 최종 제출 전에는 실제 코드와 보고서의 설명을 대조하고, 보고서 PDF 와 테스트 증거를 저장소에 올린 뒤 README 의 실행 절차와 문서 링크를 확인해야 한다.

추가 개선으로는 곡명과 아티스트를 별도 항목으로 입력하고 음악 감상 기록을 검색하는 기능을 고려할 수 있다. 이번 구현을 통해 프론트의 화면 제어와 서버의 인증 및 권한 검사가 각각 필요하다는 점을 이해했다. 또한 기능이 화면에서 동작하는지 뿐 아니라 실제 HTTP 상태와 오류 코드, 실패 후 데이터 유지 여부까지 확인해야 한다는 점을 배웠다.

제출 확인: [√] 실제 코드와 설명 일치 [√] 테스트 직접 수행 [√] AI 활용 및 출처 기록 [√] 비밀 키와 토큰 제거

GitHub 확인: [√] 루트 README 작성 [√] 보고서 PDF 와 증거 업로드 [√] README 문서 링크 확인 [√] 교수자 접근 가능 [√] 저장소 URL 과 커밋 SHA 제출